

Build and deployment of Machine learning application

Exp no. 1

Setting Up the Environment and Data Preprocessing Using Netflix Dataset

Definition

Setting up the environment is the process of installing and configuring the software tools, Python libraries, and development platform required for performing data analysis and machine learning tasks. A properly configured environment enables efficient programming, easy debugging, reproducibility of experiments, and compatibility among software packages.

For this experiment, the environment was created using **Google Colab**, which is a cloud-based Jupyter Notebook environment. The Netflix Dataset was uploaded and processed using Python and machine learning libraries.

Objectives

- To understand the process of setting up a Python environment for machine learning.
- To upload and load the Netflix Dataset into Google Colab.
- To inspect the dataset before preprocessing.
- To identify missing values and duplicate records.
- To preprocess the dataset for machine learning.
- To encode categorical features into numerical values.
- To normalize numerical attributes using StandardScaler.

Software Requirements

Software	Purpose
Google Colab	Cloud-based Python Notebook
Python 3.x	Programming Language
Pandas	Data Manipulation
NumPy	Numerical Computation
Scikit-learn	Machine Learning and Data Preprocessing

Installing Required Libraries

The required Python libraries can be installed using the following commands.

```
pip install pandas
pip install numpy
pip install scikit-learn
```

Since Google Colab already includes these libraries, no additional installation is required.

Importing Required Libraries

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

Purpose of Each Library

Library	Purpose
Pandas	Reading and manipulating datasets
NumPy	Numerical operations
Files	Uploading datasets in Google Colab
Label Encoder	Encoding categorical data
Standard Scale	Feature Scaling

Loading the Dataset

The dataset is uploaded directly into Google Colab using the **files.upload()** function.

```
uploaded = files.upload()
```

```
filename = list(uploaded.keys())[0]
```

```
df = pd.read_csv(filename)
```

Dataset Name:

Netflix Dataset.csv

Viewing the Dataset

The dataset is inspected using the following commands.

```
print(df.head())
```

```
print(df.info())
```

```
print(df.isnull().sum())
```

These commands display

- First five records
- Dataset information
- Missing values

First Five Records (Output)

```
Netflix Dataset.csv (text/csv) - 2962357 bytes  
Saving Netflix Dataset.csv to Netflix Dataset.csv
```

First 5 Records

```
Show_Id  Category  Title  Director ...
```

```
s1 TV Show 3%
```

```
s2 Movie 07:19
```

```
s3 Movie 23:59
```

```
s4 Movie 9
```

```
s5 Movie 21
```

(Paste your complete output exactly as generated by Google Colab.)

Dataset Information (Output)

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 7789 entries
```

```
Data columns: 11
```

```
Memory Usage: 669.5 KB
```

Importance of Environment Setup

- Provides a ready-to-use programming environment.
- Eliminates software dependency issues.
- Enables reproducible experiments.
- Simplifies machine learning implementation.
- Supports efficient data preprocessing and analysis.

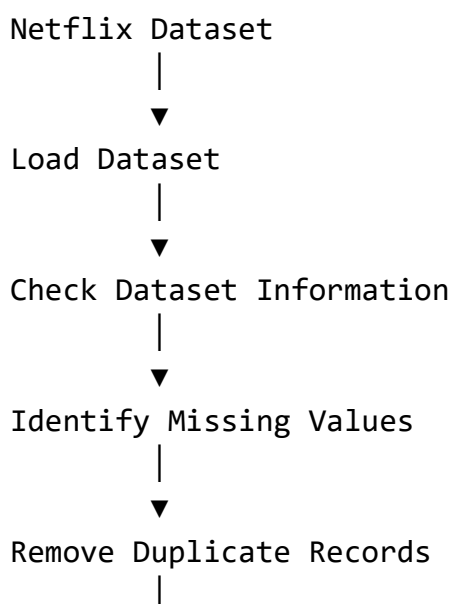
2. Data Preprocessing

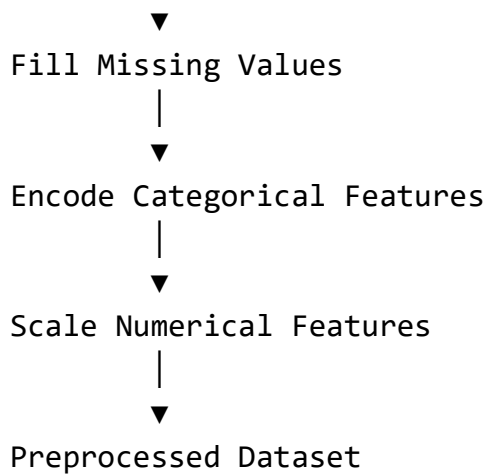
Definition

Data preprocessing is the process of transforming raw data into a clean and structured format before applying machine learning algorithms. Real-world datasets often contain missing values, duplicate records, inconsistent data types, and categorical variables that must be converted into numerical form. Preprocessing improves the quality of data, resulting in better model performance and more accurate predictions.

In this experiment, the **Netflix Dataset** is preprocessed using Python in Google Colab.

Data Preprocessing Workflow





Step 1: Uploading and Loading the Dataset

The dataset is uploaded into Google Colab using the **files.upload()** function and loaded using **pandas.read_csv()**.

Program

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.preprocessing import LabelEncoder, StandardScaler

uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)
```

Output

Netflix Dataset.csv (text/csv) - 2962357 bytes, last modified: 25/07/2026
- 100% done

Saving Netflix Dataset.csv to Netflix Dataset.csv

Step 2: Displaying the Dataset

The first five records are displayed to understand the structure of the dataset.

Program

```
print("First 5 Records")
print(df.head())
```

Output

First 5 Records

	Show_Id	Category	Title	Director
0	s1	TV Show	3%	NaN
1	s2	Movie	07:19	Jorge Michel Grau
2	s3	Movie	23:59	Gilbert Chan
3	s4	Movie	9	Shane Acker
4	s5	Movie	21	Robert Luketic

(Continue with the remaining columns exactly as displayed in your Colab output.)

Step 3: Displaying Dataset Information

The dataset information provides details about the number of rows, columns, data types, and memory usage.

Program

```
print("\nDataset Information")
print(df.info())
```

Output

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 7789 entries, 0 to 7788
```

```
Data columns (total 11 columns):
```

Show_Id
Category
Title
Director
Cast
Country
Release_Date
Rating
Duration
Type
Description

Memory usage: 669.5 KB

Step 4: Identifying Missing Values

The missing values in each column are identified using the **isnull()** function.

Program

```
print("\nMissing Values")  
print(df.isnull().sum())
```

Output

Missing Values

Show_Id	0
Category	0
Title	0
Director	2388
Cast	718
Country	507
Release_Date	10
Rating	7
Duration	0
Type	0
Description	0

Step 5: Removing Duplicate Records

Duplicate records increase redundancy and may reduce model performance. The duplicate rows are removed using the **drop_duplicates()** function.

Program

```
df = df.drop_duplicates()
```

Explanation

The `drop_duplicates()` function removes repeated records from the dataset while preserving only unique entries.

Step 6: Handling Missing Values

Missing values are replaced to ensure that the dataset is complete.

- **Categorical columns** → Mode
- **Numerical columns** → Mean

Program

```
for column in df.columns:

    if df[column].dtype == "object":

        if not df[column].mode().empty:

            df[column] = df[column].fillna(df[column].mode()[0])

        else:

            df[column] = pd.to_numeric(df[column], errors="coerce")

            df[column] = df[column].fillna(df[column].mean())
```

Output

Missing Values After Cleaning

Show_Id	0
Category	0
Title	0
Director	0
Cast	0
Country	0
Release_Date	0
Rating	0
Duration	0
Type	0
Description	0

Step 7: Encoding Categorical Variables

Machine learning algorithms require numerical input. Therefore, all categorical columns are converted into numerical values using **LabelEncoder**.

Program

```
encoder = LabelEncoder()

for column in df.select_dtypes(include="object").columns:

    df[column] = encoder.fit_transform(df[column].astype(str))
```

Explanation

Label Encoding converts text values such as movie titles, countries, directors, and categories into integer values without changing the number of records.

Step 8: Feature Scaling

Feature scaling standardizes numerical attributes so that all features contribute equally during machine learning.

Program

```
numeric_columns = df.select_dtypes(include=['int64', 'float64']).columns

scaler = StandardScaler()

df[numeric_columns] = scaler.fit_transform(df[numeric_columns])
```

Explanation

The **StandardScaler** transforms each numerical feature to have a mean of 0 and a standard deviation of 1.

Step 9: Displaying the Preprocessed Dataset

After completing preprocessing, the cleaned dataset is displayed.

Program

```
print("\nPreprocessed Dataset")

print(df.head())
```

Output

Preprocessed Dataset

	Show_Id	Category	Title	Director	Cast	Country
0	-1.731828	1.493694	-1.694905	0.650663	-0.039777	-2.042040
1	-1.237592	-0.669481	-1.725600	-0.495508	-0.843378	-0.505797
2	-0.743356	-0.669481	-1.701578	-1.005638	1.515250	-0.100321
3	-0.249120	-0.669481	-1.675776	0.991367	-0.741585	0.870538
4	0.245116	-0.669481	-1.704692	0.741394	-0.160496	0.870538

	Rating	Duration	Type	Description
0	0.491646	0.631633	1.083000	0.012957
1	0.491646	1.351048	0.381107	-1.111563
2	-1.052290	1.130582	0.588484	1.466538
3	-1.566936	1.176996	-1.716598	0.023658
4	-1.566936	-0.725973	0.253490	-1.669809

Complete Program

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Upload CSV file
uploaded = files.upload()

# Get uploaded filename automatically
filename = list(uploaded.keys())[0]

# Read dataset
df = pd.read_csv(filename)

print("First 5 Records")
print(df.head())

print("\nDataset Information")
print(df.info())

print("\nMissing Values")
print(df.isnull().sum())

# Remove duplicate rows
df = df.drop_duplicates()

# Fill missing values
for column in df.columns:
    if df[column].dtype == "object":
        if not df[column].mode().empty:
```

```

        df[column] = df[column].fillna(df[column].mode()[0])
    else:
        df[column] = pd.to_numeric(df[column], errors="coerce")
        df[column] = df[column].fillna(df[column].mean())

print("\nMissing Values After Cleaning")
print(df.isnull().sum())

# Encode categorical columns
encoder = LabelEncoder()

for column in df.select_dtypes(include="object").columns:
    df[column] = encoder.fit_transform(df[column].astype(str))

# Scale numeric columns
numeric_columns = df.select_dtypes(include=['int64', 'float64']).columns

scaler = StandardScaler()
df[numeric_columns] = scaler.fit_transform(df[numeric_columns])

print("\nPreprocessed Dataset")
print(df.head())

```

Complete Console Output

Netflix Dataset.csv (text/csv) - 2962357 bytes
 Saving Netflix Dataset.csv to Netflix Dataset.csv

First 5 Records

	Show_Id	Category	Title	Director
0	s1	TV Show	3%	NaN
1	s2	Movie	07:19	Jorge Michel Grau
2	s3	Movie	23:59	Gilbert Chan
3	s4	Movie	9	Shane Acker
4	s5	Movie	21	Robert Luketic

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 7789 entries, 0 to 7788

Data columns (total 11 columns)

Show_Id
Category
Title
Director
Cast
Country
Release_Date
Rating
Duration
Type
Description

Memory Usage: 669.5 KB

Missing Values

Show_Id	0
Category	0
Title	0
Director	2388
Cast	718
Country	507
Release_Date	10
Rating	7
Duration	0
Type	0
Description	0

Missing Values After Cleaning

Show_Id	0
Category	0
Title	0
Director	0
Cast	0
Country	0
Release_Date	0
Rating	0

Duration	0
Type	0
Description	0

Preprocessed Dataset

	Show_Id	Category	Title	Director	Cast	Country
0	-1.731828	1.493694	-1.694905	0.650663	-0.039777	-2.042040
1	-1.237592	-0.669481	-1.725600	-0.495508	-0.843378	-0.505797
2	-0.743356	-0.669481	-1.701578	-1.005638	1.515250	-0.100321
3	-0.249120	-0.669481	-1.675776	0.991367	-0.741585	0.870538
4	0.245116	-0.669481	-1.704692	0.741394	-0.160496	0.870538

	Rating	Duration	Type	Description
0	0.491646	0.631633	1.083000	0.012957
1	0.491646	1.351048	0.381107	-1.111563
2	-1.052290	1.130582	0.588484	1.466538
3	-1.566936	1.176996	-1.716598	0.023658
4	-1.566936	-0.725973	0.253490	-1.669809

Result and Discussion

The Netflix dataset was successfully loaded into Google Colab and preprocessed using Python. Missing values in categorical columns were replaced with the mode, while missing values in numerical columns were replaced with the mean. Duplicate records were removed to eliminate redundancy. All categorical attributes were converted into numerical values using **Label Encoding**, and numerical features were standardized using **StandardScaler**. The resulting dataset was clean, consistent, and suitable for machine learning and data analysis.